

Documento de Orientação — Atualização da Proposta de Pesquisa

Iniciação Científica: Aprendizado de Máquina como Functores entre Categorias

Kauã Santana da Silva — Alexandre Garcia de Oliveira — FATEC Rubens Lara

1. PROPÓSITO DESTE DOCUMENTO

Este documento registra o estado atual da pesquisa em relação à proposta original (outubro/2025), identificando o que foi cumprido, o que foi expandido além do proposto, e o que ainda está pendente. Serve de base para a atualização da proposta para submissão de bolsa, sem necessidade de reescrita do zero.

2. COMPARAÇÃO COM OS OBJETIVOS ESPECÍFICOS ORIGINAIS

(a) Revisar a literatura sobre TdC, destacando aplicações em modelagem matemática e sistemas computacionais

Status: Cumprido.

A revisão foi realizada com foco nos fundamentos necessários para a implementação: categorias, functores, categorias monoidais simétricas e composição. As referências consultadas incluem Mac Lane (1998), Awodey (2010), Mariano & Meleiro (2021) e Fong et al. (2019). O estudo foi aprofundado além do previsto: a pesquisa chegou a estudar estruturas de tipos dependentes em Haskell (GADTs, type families, DataKinds) como representação computacional das estruturas categóricas.

(b) Estudar artigos sobre a relação dos fundamentos da TdC com o Aprendizado de Máquina

Status: Cumprido e expandido.

O artigo base de Fong, Spivak & Tuyéras (2019) foi estudado em profundidade, com leitura detalhada das definições formais da categoria **Learn**, do funtor $\mathcal{L}_{\varepsilon, e}$ e da estrutura monoidal simétrica proposta. O estudo foi além do que a proposta previa: foram identificadas as limitações do artigo base (em particular, a não-injetividade de type families e a dificuldade de provar associatividade de $(++)$ em GHC 9.6) e estratégias de contorno foram desenvolvidas e documentadas.

Referência adicional identificada durante o desenvolvimento, não prevista na proposta original:

- Eisenberg (2016) — *Dependent Types in Haskell: Theory and Practice*
 - Peyton Jones et al. (2006) — *Simple Unification-based Type Inference for GADTs*
 - Yorgey et al. (2012) — *Giving Haskell a Promotion*
 - Chakravarty et al. (2005) — *Associated Types with Class*
 - Stolarek, Peyton Jones, Eisenberg (2015) — *Injective Type Families for Haskell*
-

(c) Investigar como derivadas, gradientes e otimização podem ser descritos como funtores diferenciais

Status: Cumprido e implementado.

A investigação resultou na implementação concreta do funtor $\mathcal{L}_{\varepsilon, e} : \mathbf{Para} \rightarrow \mathbf{Learn}$ com erro quadrático. As derivadas de cada morfismo foram calculadas explicitamente para cada modelo implementado:

- Regressão linear: $\partial E / \partial w = (wa + b - y) \cdot a, \partial E / \partial b = wa + b - y, \partial E / \partial a = (wa + b - y) \cdot w$
- Padronização: $\partial I_z / \partial a = 1 / \sigma$
- Ajuste quadrático: $\partial(a^2) / \partial a = 2a$
- Sigmoid com entropia cruzada: $\partial L / \partial z = \sigma(z) - y$

A distinção entre função de erro acoplada à camada de saída e gradiente propagado pelas camadas internas foi formalizada como princípio de design da implementação.

(d) Propor uma formulação categórica do aprendizado, representando-o como functor diferenciável

Status: Cumprido — com modificação em relação à proposta original.

A formulação foi implementada com uma modificação significativa: o espaço de parâmetros foi representado como **lista heterogênea tipada** $(\text{Params } (ps :: [\text{Type}]))$, em lugar do produto cartesiano usado no artigo original. Essa escolha foi motivada pelo fato de que pares em Haskell não são associativos no nível de tipos $((p, (q, r)) \neq ((p, q), r))$, o que impedia a verificação de associatividade pelo compilador.

A formulação resultante representa o espaço de parâmetros como um **monóide livre** $(\text{' []}, ++)$, onde:

- O espaço vazio (' []) é o elemento neutro
- A concatenação $(++)$ (type family) é a operação binária
- A composição $g \circ f$ é parametrizada por $(ps ++ qs)$, carregando no nível de tipos a informação completa de cada componente

Esta é a principal contribuição técnica da pesquisa em relação ao artigo base.

(e) Desenvolver um algoritmo experimental que implemente os princípios da formulação categórica

Status: Cumprido e substancialmente expandido.

O "algoritmo experimental" previsto na proposta foi superado pela implementação de uma **biblioteca categórica completa** em Haskell, com os seguintes módulos:

Core (fundamentos):

- `Core.Cat` — typeclass `Cat` com `id` e `(.)` parametrizados por `[Type]`
- `Core.Learner` — morfismo `Learner ps a b` como instância de `Cat`
- `Core.Params` — GADT `Params (ps :: [Type])` com `projectFirst`, `projectRest`, `unify`

Modelos implementados e validados:

- `Models.LinearRegressor` — regressão linear simples ($\hat{y} = wa + b$), erro quadrático
- `Models.StandardRegressor` — composição com padronização z-score pré-composta
- `Models.PolynomialRegressor` — composição com ajuste quadrático ($\hat{y} = wa^2 + b$)
- `Models.LogisticRegressor` — composição com sigmoid e entropia cruzada binária
- `Models.Net` — rede neural mínima com camada oculta (em desenvolvimento)

Sandbox (morfismos primitivos para composição livre):

- `Sandbox.Layers` — camadas densas agnósticas à função de perda
- `Sandbox.Activations` — relu, sigmoid interno
- `Sandbox.Losses` — mseOutput, bceOutput (morfismos de saída com perda embutida)
- `Sandbox.Normalizers` — z-score, min-max

Composição paralela:

- Operador `(//)` implementado em `Core.Learner`
- `Sandbox.Routing` — morfismos `comonoid` (δ) e `monoid` (μ)

Treinamento e avaliação:

- `Training.Training` — `step`, `train`, `debug`
- `Training.Accuracy` — acurácia sobre conjunto de teste
- `Dataset.*` — dados sintéticos e loader para Iris PCA

(f) Avaliar o comportamento do algoritmo em cenários controlados

Status: Parcialmente cumprido — com resultados concretos.

Os cenários de avaliação foram ampliados em relação ao previsto:

Cenários controlados (dados sintéticos):

Modelo	Dados	Épocas	ϵ	w obtido	w esperado	erro
Linear simples	$y = 2x + 1,$ $x \in [1, 16]$	1000	0,006	2,012	2,000	0,6%
Padronizado	mesmo	1000	0,01	—	—	predição $f(17) =$ 35, 24 vs 35
Polinomial	$y = 0, 3x^2 - 3$	100	0,01	0,296	0,300	1,4%
Logístico	dados binários sintéticos	1000	0,001	—	—	convergindo

Cenário real (dataset Iris com PCA):

- PCA aplicado com Python (scikit-learn), redução para 1 componente principal
- Separação treino/teste (80/20, estratificado)
- Rede neural mínima testada no conjunto PCA
- Acurácia de 100% no conjunto de teste obtida

Verificação das leis categoriais:

- `test/Testes.hs` verifica empiricamente identidade à esquerda, identidade à direita e associatividade para os modelos implementados

O que ainda não foi avaliado formalmente:

- Curva de perda por época (métrica planejada, não implementada)
- Matriz de confusão completa para o logístico
- Avaliação formal da rede neural com Iris (em andamento)

(g) Elaborar síntese teórica e prática dos resultados

Status: Parcialmente cumprido — material produzido, síntese final pendente.

Material já produzido:

- **Artigo científico** submetido ao Simpósio de Ciência de Dados da FATEC (maio/2026): "Regressões Categóricas: implementação e modelagem em Haskell baseada em *Backprop as a Functor*"
- **Apresentação** no Simpósio de Ciência de Dados (maio/2026)
- **Relatório de plano de férias** com direções de desenvolvimento

O artigo aborda os três primeiros modelos (linear, padronizado, polinomial), a modificação do espaço de parâmetros e os desafios de implementação em Haskell.

3. O QUE FOI FEITO ALÉM DO PROPOSTO

3.1 Modificação do espaço de parâmetros

A proposta original não previa dificuldades com o produto cartesiano. A pesquisa identificou que pares em Haskell quebram a associatividade no nível de tipos e desenvolveu a solução com listas heterogêneas — uma contribuição técnica não prevista que fortalece a fidelidade da implementação à estrutura categórica.

3.2 Typeclass Cat própria

A proposta previa implementar a formulação categórica sem especificar como. A pesquisa identificou que a `Category` nativa de Haskell não suporta morfismos cujo domínio muda sob composição (como ocorre em **Learn**) e desenvolveu uma typeclass própria para contornar isso.

3.3 Módulo Sandbox

Não previsto na proposta. A ideia surgiu durante o desenvolvimento como forma de demonstrar concretamente a composicionalidade: morfismos primitivos soltos que o usuário compõe livremente para construir qualquer modelo. A distinção entre camada interna (agnóstica à perda) e camada de saída (portadora do gradiente inicial) é uma contribuição conceitual derivada da implementação.

3.4 Composição paralela

Não prevista explicitamente na proposta. A implementação do operador `(//)` e dos morfismos de roteamento (`comonoid`, `monoid`) corresponde à seção do artigo base sobre estrutura monoidal simétrica — que a proposta original não mencionava.

3.5 Aplicação ao dataset Iris com PCA

Não previsto na proposta. Representa a transição de dados sintéticos controlados para dados reais, com integração Python-Haskell (PCA via scikit-learn, dataloader em Haskell com Cassava).

3.6 Rede neural mínima

A proposta previa "um algoritmo experimental" de escala reduzida. A pesquisa foi além: implementou uma biblioteca extensível onde a rede neural é apenas uma composição dos morfismos primitivos já existentes, sem código adicional de backpropagation.

4. O QUE AINDA NÃO FOI FEITO

Do cronograma original

Item	Prazo original	Status atual
(e) Algoritmo experimental	Jul-Set/2026	Cumprido e superado
(f) Avaliação em cenários controlados	Set-Nov/2026	Parcialmente cumprido
(g) Síntese teórica e prática	Out-Dez/2026	Artigo parcial produzido

Itens identificados durante o desenvolvimento (não previstos na proposta)

Item	Estado
Módulo de métricas (perda por época, matriz de confusão)	Planejado, não implementado
Normalização min-max no Sandbox	Planejado, não implementado
Avaliação formal da rede neural com Iris	Em andamento
Composição paralela — testes e exemplos completos	Implementação iniciada
Morfismos de roteamento completos (swap, delete)	Parcialmente implementado
Entradas múltiplas ($([Double])$ como tipo de (a))	Identificado, aguardando orientação
Operads / multicategorias para aridade variável	Estudo indicado pelo orientador
Composição paralela completa com morfismos de simetria	Em estudo
Prova formal das leis no sistema de tipos (GADT de concatenação)	Identificado como trabalho futuro
Estrutura monoidal simétrica completa	Em implementação

5. LIMITAÇÕES TÉCNICAS IDENTIFICADAS E DOCUMENTADAS

Estas limitações não estavam previstas na proposta e devem ser mencionadas na versão atualizada como contribuições ao entendimento das dificuldades de implementação:

1. Não-injetividade de type families no GHC 9.6 A type family `(++)` não é injetiva — dado `(ps ++ qs)`, o GHC não deduz `(ps)` e `(qs)` univocamente. Isso exigiu o uso de `unsafeCoerce` pontual em `(projectFirst)` e `(projectRest)`, matematicamente justificado mas invisível ao sistema de tipos.

2. Propriedades monoidais de `(++)` não verificáveis como equações Identidade direita (`(xs ++ '[] ≡ xs)`) e associatividade (`((xs ++ ys) ++ zs ≡ xs ++ (ys ++ zs))`) não podem ser adicionadas como equações à type family fechada no GHC. Isso impediu a verificação formal das leis categoriais pelo compilador, exigindo verificação empírica via `(Testes.hs)`.

3. Incompatibilidade do `(singletons)` com GHC 9.6 A biblioteca `(singletons)`, que resolveria o problema de tipo-dependência de forma limpa, é incompatível com GHC 9.6 nas versões disponíveis no Hackage. A solução escalável (GADT de concatenação com prova explícita) está identificada como trabalho futuro.

6. CONTRIBUIÇÕES ORIGINAIS DA PESQUISA EM RELAÇÃO AO ARTIGO BASE

Aspecto	Artigo base (Fong et al., 2019)	Esta pesquisa
Espaço de parâmetros	Produto cartesiano $P \times Q$	Monóide livre $(\text{' []}, ++)$
Verificação das leis	Teórica	Empírica + implementação
Backpropagation	Demonstrado como teorema	Demonstrado como emergência da composição em código executável
Modelos	Formulação abstrata	4 modelos concretos com resultados numéricos
Composicionalidade	Afirmada	Demonstrada através do módulo Sandbox
Dataset	Não aplicado	Iris com PCA
Implementação	Não fornecida	Biblioteca completa em Haskell open-source

7. MATERIAL PRODUZIDO DURANTE A PESQUISA

Tipo	Descrição	Status
Artigo	"Regressões Categóricas em Haskell"	Submetido ao Simpósio FATEC maio/2026
Apresentação	Slides Beamer para o Simpósio	Apresentado
Relatório	Plano de férias com direções de desenvolvimento	Entregue ao orientador
Código	Biblioteca <code>ic</code> em Haskell	Open-source em github.com/Okauaa/IC
Documento	CONTEXT.md — contexto para agentes de IA	Mantido e atualizado

8. DIREÇÕES ABERTAS PARA A VERSÃO ATUALIZADA DA PROPOSTA

Estas são as frentes abertas que podem ser incorporadas como objetivos específicos novos na proposta atualizada, dependendo da decisão do orientador:

Frente A — Completar a estrutura monoidal simétrica do paper base Implementar $\otimes$ completo com morfismos de simetria $(\lambda, \rho, \alpha, \sigma)$, swap e delete. Isso completa o que o paper propõe dentro de **Learn** sem sair do framework.

Frente B — Investigar operads e multicategorias para aridade variável Estudo indicado pelo orientador para o período de férias. Resolve o problema de morfismos com n entradas interagindo (regressão múltipla, PCA como learner) de forma teoricamente sólida.

Frente C — Rede neural com múltiplas camadas e avaliação formal Extensão natural do que já foi implementado, com avaliação em datasets reais (Iris completo, não apenas PCA binário) e métricas formais (curva de perda, matriz de confusão).

Frente D — Artigo de resultados O artigo submetido ao Simpósio é parcial (três modelos iniciais). Um artigo completo abrangendo a rede neural, o Sandbox, a composição paralela e os resultados com Iris seria a síntese prevista no objetivo (g) da proposta original.